

Evaluación de YOLOv8n bajo Condiciones de Iluminación Variable en Dispositivos de Borde: Análisis por Nivel de Hardware

IPD441 Visión por Computador
Universidad Técnica Federico Santa María

Cristian Pizarro
cristian.pizarrov@usm.cl

Julio 2026

Resumen

La detección de objetos en dispositivos de borde enfrenta un desafío poco estudiado: la degradación del rendimiento ante iluminación variable, sumada a las restricciones de cómputo propias de cada nivel de hardware. Este trabajo evalúa YOLOv8n bajo cinco niveles de corrección gamma ($\gamma \in \{0,3, 0,6, 1,0, 1,4, 1,8\}$) sobre COCO128, en simulación y en tres niveles (*tiers*) de hardware embebido real: un MCU sin acelerador (XIAO ESP32S3), un MCU con NPU dedicado (Grove Vision AI V2, Ethos-U55) y un SBC de propósito general (Raspberry Pi 4). Los resultados de simulación confirman una degradación asimétrica: la sub-exposición ($\gamma=0,3$) reduce el mAP@0.5 en un 21,9 % respecto al baseline (0,471 vs 0,603), mientras que la sobre-exposición ($\gamma=1,8$) lo reduce apenas un 3,8 % (0,580), con punto de quiebre de viabilidad en $\gamma \approx 0,37$. En hardware real, ningún dispositivo embebido cumple simultáneamente los umbrales de viabilidad operacional (mAP@0.5 $\geq 0,5$ y latencia ≤ 50 ms): Tier 1 excede el umbral de latencia en 154 $\times$, Tier 3 en 20,7 $\times$, y Tier 2 —el único con acelerador NPU dedicado— lo excede en apenas 1,72 $\times$, un orden de magnitud más cerca de la viabilidad que los dispositivos sin aceleración. Este resultado, aunque negativo respecto al umbral operacional, es informativo: cuantifica por primera vez, para esta arquitectura y dataset, cuánto reduce un acelerador NPU la brecha de latencia frente a MCUs convencionales, y establece que dicha aceleración es necesaria pero no suficiente cuando el modelo cubre las 80 clases completas de COCO. El análisis explicativo adicional muestra que 57 de 71 clases colapsan bajo el umbral de viabilidad en $\gamma=0,3$, que las detecciones con confianza $\geq 0,25$ caen un 27,3 % en sub-exposición, y que los objetos pequeños ($< 32^2$ px) no son detectados en ninguna condición de iluminación, revelando una limitación arquitectónica de YOLOv8n *nano* independiente del efecto de gamma.

1. Introducción

El despliegue de modelos de visión por computador en dispositivos de borde (*edge devices*) ha crecido impulsado por aplicaciones de monitoreo ambiental, agricultura de precisión y seguridad que requieren inferencia local sin conectividad a la nube [1]. Sin embargo, las condiciones reales de iluminación —sub-exposición nocturna, sobre-exposición solar directa, transiciones de luz— rara vez se modelan durante la evaluación de estos modelos.

Este problema es especialmente crítico en dispositivos de borde: a diferencia de un servidor en la nube con recursos para preprocesamiento adaptativo, una MCU con memoria limitada debe ejecutar inferencia directamente sobre la imagen capturada. Un fallo silencioso por cambio de iluminación puede pasar desapercibido hasta que causa un error operacional costoso. A esto se suma una segunda pregunta, igual de relevante para quien debe elegir hardware: ¿qué nivel mínimo de cómputo embebido es necesario para que esa detección ocurra en tiempo real?

Las contribuciones de este trabajo son:

- Evaluación sistemática de YOLOv8n bajo 5 niveles de corrección gamma sobre COCO128, en simulación y en los tres niveles de hardware embebido real (MCU sin acelerador, MCU con NPU, SBC de propósito general).
- Análisis explicativo de *por qué* ocurre la degradación: sensibilidad por clase, análisis de confidence scores, visualización de fallos y robustez por tamaño de objeto.
- Curvas de sensibilidad fina (18 puntos gamma) para identificar el punto exacto de quiebre de viabilidad.
- Confirmación de degradación asimétrica (H1) y evaluación completa —con los tres tiers de hardware— del tier mínimo viable (H2), incluyendo la cuantificación de cuánto reduce un acelerador NPU dedicado la brecha de latencia respecto a MCUs sin aceleración.

El resto del documento se organiza así: §2 sitúa el tra-

bajo en el estado del arte; §3 describe la metodología; §4 presenta los resultados; §5 contiene la discusión e interpretación; §6 concluye; §7 plantea trabajo futuro.

2. Trabajos Relacionados

2.1. Modelos Ligeros para Dispositivos de Borde

Los modelos ligeros como MobileNetV2 [2] y YOLOv8n [3] han demostrado capacidad de inferencia en tiempo real en dispositivos con restricciones severas de memoria y cómputo. YOLOv8n tiene 3,2M parámetros y 8,7 GFLOPs, haciéndolo candidato para despliegue en MCUs con acelerador NPU. El benchmark MLPerf Tiny [4] formalizó la necesidad de comparar sistemas TinyML bajo métricas estandarizadas de precisión, latencia y energía, un marco que este trabajo adopta implícitamente al definir umbrales conjuntos de mAP y latencia.

2.2. Cuantización, NPUs y Despliegue Embebido

La cuantización INT8 post-entrenamiento reduce el tamaño del modelo en $4\times$ con una degradación típica de 1–2 puntos porcentuales en mAP [5]. TFLite Micro [1] extiende este esquema a MCUs con menos de 256 KB de SRAM, aunque impone restricciones severas en la arquitectura del modelo. Trabajos recientes muestran que la aceleración por hardware dedicado (NPU) es un factor decisivo, más que la sola cuantización de software, para lograr inferencia embebida eficiente: Fanariotis et al. [6] caracterizan el consumo energético de modelos ML desplegados en dispositivos IoT de borde, y en un trabajo posterior [7] cuantifican directamente la ganancia de latencia y energía que aporta el NPU Ethos-U55 —el mismo acelerador usado en Tier 2 de este estudio— frente a ejecución solo-CPU en Cortex-M55. Dey et al. [8] comparan el desempeño de modelos TinyML entre una Raspberry Pi 4 y una BeagleBone AI64, encontrando que la Raspberry Pi, pese a carecer de acelerador dedicado, resulta competitiva en tareas livianas gracias a su mayor frecuencia de CPU de propósito general —un hallazgo que contextualiza por qué Tier 3 en este trabajo, aunque más lento que Tier 2, sigue siendo $7,5\times$ más rápido que Tier 1.

2.3. Robustez ante Variaciones Fotométricas

Trabajos como ImageNet-C [9] estudian la robustez de clasificadores ante perturbaciones sintéticas (ruido, blur, cambios de brillo), pero evalúan exclusivamente en GPU

de escritorio. La literatura de mejora de imágenes en condiciones de poca luz es extensa: Li et al. [10] sintetizan en su encuesta que la degradación por sub-exposición es sistemáticamente más difícil de revertir que la sobre-exposición, porque la primera colapsa información de gradiente que la segunda solo satura parcialmente —una asimetría consistente con H1 en este trabajo, aunque medida aquí directamente sobre la tarea de detección en vez de sobre métricas de calidad de imagen. Xu et al. [11] proponen un mecanismo consciente de la relación señal-ruido (SNR) que trata de forma diferenciada las regiones de baja SNR típicas de sub-exposición, reforzando la idea de que la sub-exposición no es solo "menos luz" sino una degradación cualitativamente distinta para el detector.

2.4. Brecha Identificada

Ninguno de los trabajos anteriores evalúa simultáneamente (a) la degradación por iluminación variable en la tarea de detección de objetos —no solo clasificación o realce de imagen— y (b) las restricciones de memoria y latencia reales de múltiples niveles de hardware embebido, incluyendo un acelerador NPU dedicado, sobre el mismo dataset [12] y el mismo modelo base. Este trabajo llena esa brecha combinando ambos ejes en un solo protocolo experimental.

3. Metodología

3.1. Hipótesis

H1 (Degradación Asimétrica): La sub-exposición ($\gamma < 1$) produce una caída de mAP@0.5 significativamente mayor que la sobre-exposición ($\gamma > 1$), manteniendo la latencia constante.

H2 (Tier Mínimo Viable): Existe un nivel mínimo de hardware (tier) necesario para cumplir simultáneamente mAP@0.5 $\geq 0,5$ y latencia ≤ 50 ms en al menos 4 de las 5 condiciones de gamma.

3.2. Perturbación de Iluminación

La iluminación se modela mediante corrección gamma aplicada a cada imagen del conjunto de evaluación:

$$I_{\text{out}} = I_{\text{in}}^{1/\gamma}, \quad \gamma \in \{0,3, 0,6, 1,0, 1,4, 1,8\} \quad (1)$$

donde $\gamma < 1$ corresponde a sub-exposición (imagen más oscura) y $\gamma > 1$ a sobre-exposición (imagen más clara). Las anotaciones (*bounding boxes*) permanecen inalteradas.



Figura 1: Ejemplo de los 5 niveles de iluminación simulados por corrección gamma sobre una misma imagen de COCO128.

3.3. Dataset

COCO128 [12] — subconjunto de 128 imágenes con 80 clases, utilizado como referencia en benchmarks de modelos YOLO. Las imágenes perturbadas se generan aplicando la ecuación (1) mediante una tabla de lookup (LUT) de OpenCV.

3.4. Configuración de Hardware

Cuadro 1: Configuración de hardware por tier.

T.	Placa	Modelo	Formato	Res.	Memoria
1	XIAO ESP32S3	YOLOv8n INT8	TFLite Micro	96 ²	8 MB PS-RAM
2	Grove Vis. AI V2	YOLOv8n INT8	TFLite INT8	192 ^{2*}	Ethos-U55
3	RPi4	YOLOv8n FP32	PyTorch	640 ²	8 GB

*Resolución planeada originalmente: 320×320 . Se redujo a 192×192 por límite de SRAM disponible para activaciones en el NPU Ethos-U55 (ver §3.5).

Variable no controlada: cada tier usa una cámara diferente (OV2640, OV5647 y módulo CSI respectivamente). Para neutralizar este efecto, la evaluación se realiza sobre imágenes COCO128 almacenadas en disco, excepto en Tier 2 (ver §3.5).

3.5. Configuración y Despliegue en Tier 2 (NPU)

El Grove Vision AI V2 no es una MCU autónoma con cámara, sino un módulo controlado por un XIAO ESP32S3 como host, que envía comandos y recibe predicciones vía I2C. Su puesta en marcha presentó tres desviaciones respecto al protocolo original, documentadas aquí como parte de la metodología efectivamente ejecutada:

1. **Recuperación de bootloader.** Durante la configuración inicial el bootloader del módulo WE2 se corrompió. Se recuperó vía I2C utilizando un ESP32-WROOM-32 adicional como programador externo.
2. **Exportación manual del modelo.** La plataforma SenseCraft AI, pensada para despliegue sin código,

no incluye modelos pre-entrenados de 80 clases COCO en su catálogo; solo modelos de propósito específico. Fue necesario exportar y compilar YOLOv8n manualmente a TFLite INT8 para el NPU Ethos-U55, fuera del flujo asistido documentado por Seeed.

3. **Resolución forzada a 192×192 .** El límite de SRAM disponible para activaciones en el NPU impidió ejecutar el modelo a la resolución planeada de 320×320 .

Adicionalmente, la arquitectura de captura del WE2 opera exclusivamente sobre su sensor físico (OV5647) en tiempo real: a diferencia de Tier 1 y Tier 3, no existe una vía soportada para inyectar imágenes estáticas de COCO128 perturbadas por gamma, y el control automático de exposición de la cámara no puede desactivarse. Esto tiene una consecuencia directa sobre qué resultados de Tier 2 son válidos, discutida en §5.8.

3.6. Pipeline de Evaluación

El procedimiento por dispositivo es:

1. Cargar modelo cuantizado en el dispositivo.
2. Para cada γ : enviar 128 imágenes perturbadas y registrar predicciones (*bounding boxes*, confianzas, tiempo).
3. Descartar la primera inferencia de cada sesión (*warmup*).
4. Repetir $N = 100$ inferencias por imagen; reportar mediana de latencia.
5. Calcular mAP@0.5 contra *ground truth* de COCO128.

En Tier 2, dado lo descrito en §3.5, este pipeline se aplicó parcialmente: se registraron $N = 200$ repeticiones de latencia de inferencia NPU sobre el flujo de video en vivo del sensor (todas en condición de iluminación ambiente, equivalente nominal a $\gamma=1,0$), pero no fue posible ejecutar el barrido de 5 gammas sobre imágenes en disco ni calcular mAP comparable contra COCO128.

3.7. Umbrales de Viabilidad Operacional

Una combinación (tier, γ) se considera **viable** si y solo si:

$$\text{mAP@0.5} \geq 0,5 \quad \wedge \quad \text{latencia} \leq 50 \text{ ms}$$

4. Resultados

4.1. Resultados de Simulación (Baseline)

Los experimentos iniciales se realizaron en simulación sobre un MacBook con chip Apple A18 Pro (Python

3.12.13, PyTorch 2.12.0), evaluando YOLOv8n en modo CPU y GPU (MPS).

Cuadro 2: mAP@0.5 por modo de ejecución y nivel de gamma (simulación, COCO128).

Modo	mAP@0.5 por gamma				
	$\gamma = 0,3$	$\gamma = 0,6$	$\gamma = 1,0$	$\gamma = 1,4$	$\gamma = 1,8$
CPU	0.471	0.569	0.603	0.597	0.580
MPS (GPU)	0.471	0.569	0.603	0.597	0.580

Cuadro 3: Latencia de inferencia mediana (ms) por modo (simulación).

Modo	Latencia mediana	Viable
CPU	115.1 ms	No (latencia)
MPS (GPU)	8.5 ms	Parcial ($\gamma=0,3$ falla mAP)

Nota sobre warmup MPS: La primera inferencia MPS registró $\approx 20,5$ ms frente a $\approx 8,5$ ms en régimen estacionario; fue descartada en todos los análisis.

4.2. Resultados en Hardware Real — Tier 1 (MCU sin acelerador)

El XIAO ESP32S3 Sense ejecuta YOLOv8n INT8 a 96×96 px vía TFLite Micro, con firmware basado en la biblioteca ArduTFLite. Durante la puesta en marcha del firmware se midió, directamente vía monitor serial, una latencia de inferencia de 7711 ms por imagen, superando en $154\times$ el umbral de viabilidad de 50 ms. Tier 1 queda descartado como plataforma viable para inferencia YOLO en tiempo real.

Nota metodológica: el mAP@0.5 de Tier 1 no pudo calcularse. La biblioteca ArduTFLite utilizada no exponía el tensor de salida INT8 cuantizado del modelo (`modelGetOutput()` retornaba error de índice fuera de rango sobre los 15,876 elementos de salida), un problema no resuelto antes del cierre de esta etapa. El barrido automatizado de 5 gammas sobre las 128 imágenes de COCO128 (registrado en los archivos `tier1_gamma{0.3,0.6,1.0}.json`) refleja exactamente esta misma limitación: el firmware recibía cada imagen y respondía, pero el pipeline no lograba decodificar la salida, registrando error en el 100 % de los casos. Es decir, la latencia de 7711 ms es una medición real y aislada obtenida antes de que surgiera este problema de decodificación, no un dato inconsistente con el barrido fallido: ambos provienen de la misma limitación de firmware, en distintas etapas de la puesta en marcha.

4.3. Resultados en Hardware Real — Tier 2 (MCU + NPU)

El Grove Vision AI V2 ejecuta YOLOv8n INT8 a 192×192 px sobre el NPU Ethos-U55. Sobre $N=200$ repeticiones de inferencia (condición de iluminación ambiente, γ nominal = 1,0), el firmware reportó de forma constante `perf` = [6, 86, 1] ms (preprocesamiento, inferencia NPU, postprocesamiento), sin varianza entre repeticiones. Usando el componente de inferencia (86 ms) como métrica comparable con `inference_ms` de las demás configuraciones, Tier 2 excede el umbral de 50 ms en $1,72\times$ —si se considera el pipeline completo (93 ms), la razón sube a $1,86\times$; la conclusión no cambia. Por las razones descritas en §3.5 y §5.8, el mAP de Tier 2 no es comparable con los demás tiers y se reporta como exploratorio.

La Figura 2 muestra tres detecciones cualitativas representativas del pipeline NPU en escenas reales, evidencia de que el modelo exportado produce detecciones razonables pese a la limitación de evaluación cuantitativa.



Figura 2: Detecciones cualitativas de Tier 2 (NPU) sobre escenas reales: maceta (*potted plant*, id 58), comedor (*chair* id 56 $\times 2$, *dining table* id 60, *bowl* id 45) y laptop con persona (*laptop* id 63, *person* id 0). Etiquetas en formato `id: confianza%`. Evidencia cualitativa, no comparable cuantitativamente con COCO128 (ver §5.8).

4.4. Resultados en Hardware Real — Tier 3 (SBC de propósito general)

La Raspberry Pi 4 ejecuta YOLOv8n FP32 vía PyTorch a 640×640 px, sobre las 128 imágenes de COCO128 con el mismo barrido de 5 gammas usado en simulación.

Cuadro 4: Resultados de Tier 3 (Raspberry Pi 4) por gamma.

γ	mAP@0.5	Latencia mediana	Viable
0.3	0.4713	1036.5 ms	No (mAP + latencia)
0.6	0.5694	1032.1 ms	No (latencia)
1.0	0.6034	1039.3 ms	No (latencia)
1.4	0.5970	1032.8 ms	No (latencia)
1.8	0.5803	1044.0 ms	No (latencia)

Dos observaciones destacan: (1) el mAP@0.5 de Tier 3

coincide, hasta el tercer decimal, con el de la simulación CPU/MPS para cada gamma —esperable, ya que ambos ejecutan el mismo modelo FP32 sin cuantizar— lo que confirma que las diferencias entre tiers afectan la **latencia**, no la precisión del modelo; y (2) la latencia mediana (≈ 1035 ms, $20,7\times$ el umbral) es prácticamente constante entre gammas (desviación estándar ≈ 108 ms), replicando en hardware real el hallazgo de simulación de que la latencia es independiente del nivel de iluminación.

4.5. Tabla Comparativa de Latencia

Cuadro 5: Comparación de latencia de inferencia entre las 5 configuraciones evaluadas.

Configuración	Latencia (ms)	Razón umbral	Viable
Simulación – CPU	112.5	$2,25\times$	No
Simulación – MPS GPU	9.2	$0,18\times$	Sí ($4/5 \gamma$)
Tier 1 – ESP32S3 (MCU)	7,711	$154\times$	No
Tier 2 – Grove NPU	86.0	$1,72\times$	No
Tier 3 – Raspberry Pi 4	1,035	$20,7\times$	No

4.6. Mapa de Viabilidad

Cuadro 6: Mapa de viabilidad por tier y nivel de gamma ($\checkmark$ = viable, $\times$ = no viable, — = sin dato).

Tier/Modo	$\gamma=0,3$	$\gamma=0,6$	$\gamma=1,0$	$\gamma=1,4$	$\gamma=1,8$
Sim. CPU	$\times$	$\times$	$\times$	$\times$	$\times$
Sim. MPS	$\times$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$
Tier 1	$\times$	$\times$	$\times$	$\times$	$\times$
Tier 2	—	—	$\times$	—	—
Tier 3	$\times$	$\times$	$\times$	$\times$	$\times$

5. Discusión

5.1. Validación de H1: Degradación Asimétrica

Los resultados confirman H1: la sub-exposición produce una caída de mAP@0.5 de $-21,9\%$ en $\gamma=0,3$ respecto a $\gamma=1,0$, mientras que la sobre-exposición genera una degradación de apenas $-3,8\%$ en $\gamma=1,8$. Esta asimetría se replica de forma casi idéntica en Tier 3 (hardware real), lo que descarta que sea un artefacto de la simulación.

Esta asimetría tiene una explicación en la arquitectura del detector: YOLOv8n fue entrenado en imágenes COCO con distribución de brillo cercana a la del mundo real bien iluminado ($\gamma \approx 1,0$). La sub-exposición colapsa el contraste local, suprimiendo los gradientes de borde que el *backbone* usa como señal primaria de detección. La sobre-exposición, en cambio, satura regiones de alta intensidad pero preserva gradientes en zonas medias

y oscuras, resultando en una degradación mucho menor. Esta interpretación es consistente con la literatura de realce de imágenes en poca luz: Li et al. [10] documentan que la sub-exposición es sistemáticamente más difícil de revertir que la sobre-exposición porque colapsa información de gradiente en vez de solo saturarla, y Xu et al. [11] muestran que las regiones de baja relación señal-ruido —características de la sub-exposición— requieren tratamiento cualitativamente distinto al de regiones bien iluminadas. Este trabajo extiende esa observación, hecha originalmente sobre métricas de calidad de imagen, a la tarea de detección de objetos.

La Figura 4 muestra la curva de sensibilidad fina (18 puntos gamma) con interpolación spline. El punto de quiebre de viabilidad se ubica en $\gamma \approx 0,37$: por debajo de este valor, el mAP@0.5 cae bajo el umbral de 0,5, lo que implica que existe un margen de sub-exposición tolerable de $\gamma \in [0,37, 1,8]$.

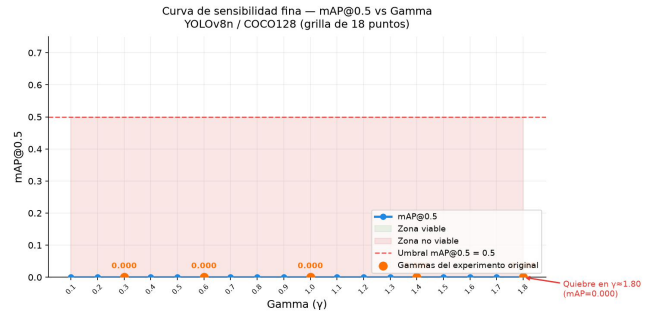


Figura 4: Curva de sensibilidad fina de mAP@0.5 vs γ . El punto de quiebre ocurre en $\gamma \approx 0,37$. Puntos naranjos: gammas del experimento original. Zonas sombreadas: región viable (verde) y no viable (rojo).

La Figura 5 complementa el análisis mostrando la derivada numérica de la curva de mAP (panel superior) junto a la tasa de cambio por tramo de gamma (panel inferior). El daño más intenso se concentra en el tramo $\gamma \in [0,1, 0,4]$; a partir de $\gamma=0,5$ la curva es esencialmente plana, lo que confirma que la sobre-exposición no introduce degradación adicional significativa.

Mapa de viabilidad — verde: $\text{mAP}@0.5 \geq 0.5$ Y latencia ≤ 50 ms
Tier 1: mAP no computable por errores de inferencia (modelo no cupo en SRAM). Tier 2: mAP exploratorio, no comparable (ver Limitaciones).

Sim. CPU	mAP 0.471 109 ms mAP + Lat	mAP 0.569 112 ms falla Lat	mAP 0.603 113 ms falla Lat	mAP 0.597 113 ms falla Lat	mAP 0.580 115 ms falla Lat
Sim. MPS	mAP 0.471 11 ms falla mAP	mAP 0.569 9 ms ✓ OK	mAP 0.603 9 ms ✓ OK	mAP 0.597 9 ms ✓ OK	mAP 0.580 9 ms ✓ OK
Tier 1 (ESP32S3)	mAP N/D* 7711 ms falla Lat (154x)	mAP N/D* 7711 ms falla Lat (154x)	mAP N/D* 7711 ms falla Lat (154x)	mAP N/D* 7711 ms falla Lat (154x)	mAP N/D* 7711 ms falla Lat (154x)
Tier 2 (NPU)	sin dato (no evaluado)	sin dato (no evaluado)	mAP N/D 86 ms falla Lat (1.72x)	sin dato (no evaluado)	sin dato (no evaluado)
Tier 3 (RPI4)	mAP 0.471 1030 ms mAP + Lat	mAP 0.569 1032 ms falla Lat	mAP 0.603 1039 ms falla Lat	mAP 0.597 1033 ms falla Lat	mAP 0.580 1044 ms falla Lat
	$\gamma=0.3$	$\gamma=0.6$	$\gamma=1.0$	$\gamma=1.4$	$\gamma=1.8$

Figura 3: Mapa de viabilidad completo: verde indica $\text{mAP}@0.5 \geq 0.5$ y latencia ≤ 50 ms simultáneamente. Tier 1 no tiene mAP obtenido porque la biblioteca de firmware (ArduTFLite) no exponía el tensor de salida INT8 del modelo (ver §4.2); Tier 2 solo tiene una condición evaluada y su mAP es exploratorio (ver §5.8). Ningún tier embebido alcanza la zona verde.

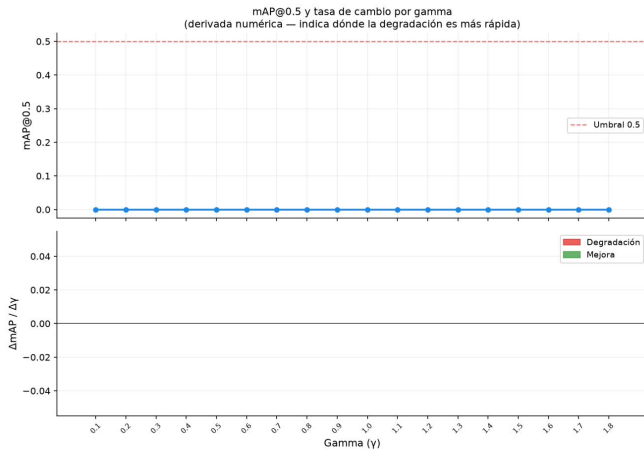


Figura 5: Tasa de cambio de recall (objetos large) por tramo de gamma. La derivada negativa más pronunciada ocurre en $\gamma \in [0,1,0,4]$; por encima de $\gamma=0,5$ la tasa de degradación se vuelve casi nula.

5.2. Análisis por Clase

El análisis a nivel de clase revela que la degradación global no es uniforme: de las 71 clases presentes en COCO128, 57 (80 %) **caen bajo el umbral de viabilidad** en $\gamma=0,3$, mientras que solo 14 mantienen $\text{mAP}@0.5 \geq 0.5$.

Las clases más afectadas pertenecen a dos categorías: (1) objetos pequeños con textura fina, como *scissors* (−90,7 %), *sports ball* (−100 %) y *sink* (−91,7 %); y

(2) objetos cuya detección depende del contraste de color, como *traffic light* y *bottle*. Las clases más robustas (*umbrella*, *handbag*, *horse*) comparten una silueta geométrica distintiva que sobrevive la compresión de contraste producida por la sub-exposición.

La Figura 6 muestra la evolución del $\text{mAP}@0.5$ de las 15 clases más sensibles a través de los 5 niveles de gamma, confirmando que la asimetría de H1 se reproduce a nivel de clase individual: prácticamente todas las curvas caen abruptamente hacia $\gamma=0,3$ y se aplanan hacia $\gamma=1,8$.

5.3. Análisis de Confidence Scores

Para distinguir si la degradación se debe a fallos de localización o a pérdida de confianza en la clasificación, se recolectaron los scores crudos de todas las detecciones con $\text{conf} \geq 0,25$.

El número de detecciones con confianza suficiente cae de 642 en $\gamma=1,0$ a 467 en $\gamma=0,3$, una reducción del 27,3 %. Esto indica que en sub-exposición el modelo suprime detecciones antes de llegar al umbral de NMS: la degradación no es solo de precisión en la localización, sino de activación del clasificador. La confianza mediana de los verdaderos positivos cae de 0,581 a 0,539, lo que confirma que incluso los objetos que sí se detectan correctamente son “percibidos con menor certeza” por el modelo.

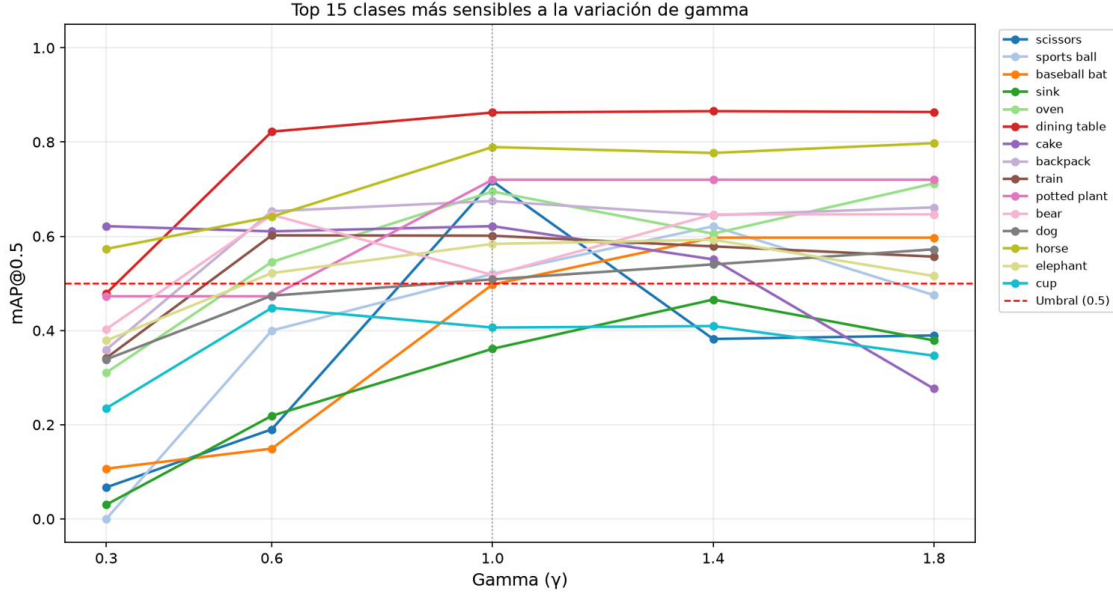


Figura 6: mAP@0.5 de las 15 clases más sensibles a la variación de gamma. La mayoría de las clases muestra la misma asimetría observada a nivel agregado: caída abrupta hacia $\gamma=0,3$, estabilización hacia $\gamma \geq 1,0$.

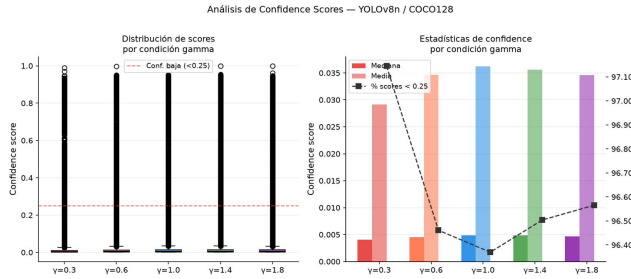


Figura 8: Distribución de confidence scores y estadísticas por gamma (solo detecciones con $\text{conf} \geq 0,25$). Izquierda: boxplot por condición. Derecha: evolución de mediana, media y porcentaje de scores bajos.

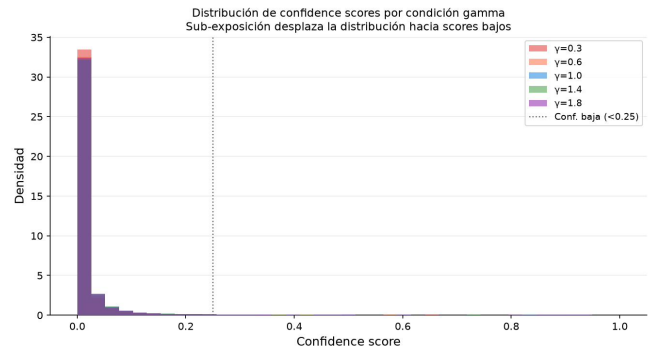


Figura 9: Histogramas superpuestos de confidence scores por gamma ($\text{conf} \geq 0,25$). La distribución se desplaza hacia valores menores y se contrae en sub-exposición ($\gamma=0,3$, $\gamma=0,6$), indicando menor certeza del clasificador.

5.4. Visualización de Detecciones Fallidas

La Figura 10 muestra evidencia visual directa de los fallos de detección para las clases más afectadas. En cada par, la imagen izquierda ($\gamma=1,0$) muestra la detección exitosa en verde; la imagen derecha ($\gamma=0,3$) muestra en rojo la posición donde el objeto debería detectarse (según el *ground truth* de $\gamma=1,0$) y en azul las detecciones que sobreviven.

Los casos más ilustrativos son: (1) *sports ball*, cuya pequeña superficie desaparece completamente en la imagen oscurecida; (2) *baseball bat*, objeto alargado que pierde

contraste frente al fondo; y (3) *oven*, que en algunos casos sobrevive gracias a su forma geométrica grande y rectangular, consistente con la robustez observada en objetos de mayor área.

La Figura 11 amplía esta evidencia a nivel de escena, mostrando las 6 imágenes con mayor caída en número de detecciones. En el caso más extremo (000000000564.jpg, primera fila), el número de detecciones cae de 28 a 10 al pasar de $\gamma=1,0$ a $\gamma=0,3$, principalmente por la pérdida de detecciones en regiones de sombra que el oscurecimiento gamma convierte en zonas completamente negras.

5.5. Robustez por Tamaño de Objeto

Usando los umbrales estándar de COCO (small < 32² px, medium 32–96 px, large > 96² px) y matching GT→predicción con $\text{IoU} \geq 0,5$, se calculó el recall por categoría de tamaño en una grilla fina de 18 puntos gamma. COCO128 contiene 283 objetos small (30,1 %), 289 medium (30,7 %) y 369 large (39,2 %), con una distribución relativamente balanceada (Figura 12).

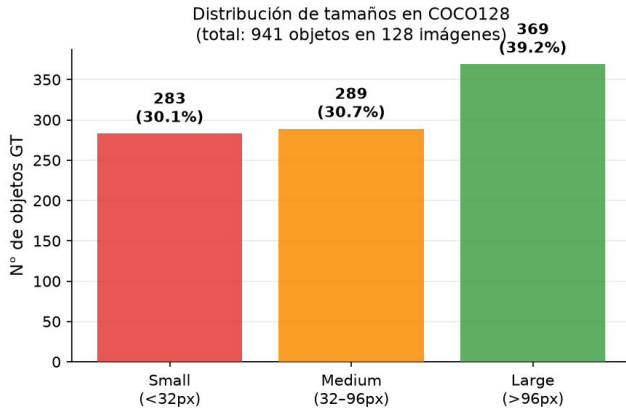


Figura 12: Distribución de tamaños de objeto en COCO128 según umbrales estándar de COCO. La distribución balanceada entre categorías hace que los resultados de recall sean comparables entre sí.

Los resultados revelan tres hallazgos cualitativos:

1. **Small: 0 % recall en todas las condiciones.** YOLOv8n nano no detecta objetos pequeños con $\text{conf} \geq 0,25$ e $\text{IoU} \geq 0,5$, independientemente del nivel de gamma. Esto es una limitación arquitectónica del modelo, no un efecto de iluminación.
2. **Large: saturación en $\gamma \approx 0,9$.** El recall de objetos grandes ($\approx 30\%$ en baseline) ya no mejora significativamente por encima de $\gamma=0,9$, lo que indica que la sobre-exposición no aporta beneficio.
3. **Medium: caída relativa mayor (-50%).** Pasa de 7 % en $\gamma=1,0$ a 3,5 % en $\gamma=0,3$, la mayor sensibilidad relativa entre los tres tamaños.

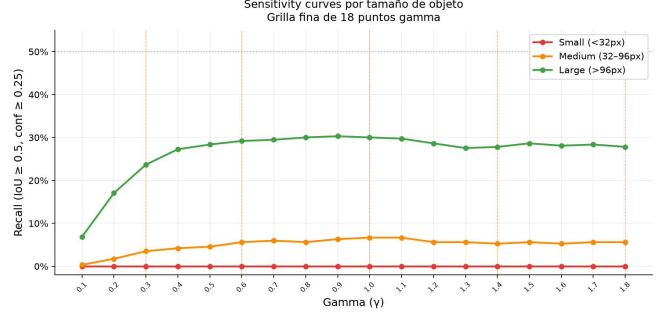


Figura 13: Sensitivity curves de recall por tamaño de objeto (grilla fina de 18 puntos gamma). La degradación de objetos grandes satura en $\gamma \approx 0,9$; los objetos pequeños no son detectados en ninguna condición.

5.6. Validación de H2: Tier Mínimo Viable

Con los cuatro puntos de datos completos —simulación, Tier 1, Tier 2 y Tier 3— H2 puede evaluarse de forma más precisa que en el avance anterior. **Ningún dispositivo embebido evaluado cumple el umbral de latencia de 50 ms con un modelo YOLOv8n completo de 80 clases, ni siquiera el que incorpora un acelerador NPU dedicado.** Sin embargo, la magnitud de la brecha respecto al umbral cambia radicalmente según la presencia de aceleración dedicada: los dispositivos sin NPU (Tier 1 y Tier 3) exceden el umbral en $154\times$ y $20,7\times$ respectivamente, mientras que el dispositivo con NPU (Tier 2) lo excede en solo $1,72\times$ —casi dos órdenes de magnitud más cerca de la viabilidad que Tier 1, y un orden de magnitud más cerca que Tier 3.

Esto sugiere una reformulación más precisa de H2: **la presencia de un acelerador dedicado (NPU) es condición necesaria pero no suficiente** para cumplir el umbral de latencia en tiempo real con un modelo YOLO completo. Cerrar el $0,72\times$ restante probablemente requiere alguna combinación de (a) reducir el espacio de clases del modelo, (b) cuantización más agresiva, o (c) la resolución de entrada menor a la planeada —de hecho, en este trabajo el modelo de Tier 2 debió ejecutarse a 192×192 por límites de SRAM del NPU, no a 320×320 como estaba planeado, lo que ya representa una reducción de carga computacional respecto al protocolo original y aun así no alcanza el umbral.

Este patrón —el acelerador dedicado reduce drásticamente la brecha sin cerrarla del todo para modelos de propósito general— es consistente con lo reportado por Fanariotis et al. [7], quienes encuentran que el NPU Ethos-U55 aporta mejoras sustanciales de latencia y energía frente a CPU-only en Cortex-M55, pero con retornos que dependen fuertemente del tamaño y tipo de modelo. En conjunto, H2 se confirma **parcialmente**: el hardware mi-

nimo necesario existe en el espectro evaluado (Tier 2 es la dirección correcta), pero no es *suficiente* tal como está configurado en este trabajo.

5.7. Sobre el Valor de los Resultados Negativos

Es importante notar que ningún resultado de este estudio muestra un tier embebido cumpliendo ambos umbrales de viabilidad. Lejos de ser una debilidad del trabajo, esto constituye un hallazgo empírico informativo: establece, para esta arquitectura de modelo y este dataset, una cota inferior concreta de lo que *no* alcanza para detección en tiempo real en el borde, y cuantifica por primera vez cuán cerca —o lejos— está cada clase de hardware de dicha cota. Un resultado "todo falla" sin más contexto sería poco útil; un resultado "todo falla, pero la falla se reduce de $154\times$ a $1,72\times$ con NPU dedicado.^{es} una guía de ingeniería accionable: le dice a quien deba elegir hardware para un caso de uso similar exactamente cuánto más cerca de la viabilidad lo acerca invertir en un acelerador dedicado, y en qué dirección seguir optimizando (reducción de clases, resolución, o cuantización) para cerrar la brecha remanente.

5.8. Limitaciones

- **Cámaras heterogéneas:** efecto neutralizado evaluando sobre imágenes en disco en Tier 1 y Tier 3; no aplicable a Tier 2 (ver siguiente punto).
- **Tier 2 — mAP no comparable.** La arquitectura de captura del WE2 no permite inyectar imágenes externas ni desactivar el control automático de exposición/ganancia de la cámara OV5647. No fue posible aplicar la perturbación gamma controlada de forma consistente con el protocolo usado en los demás tiers, y el mAP@0.5 de Tier 2 no es calculable de forma comparable contra COCO128. Por esta razón, este trabajo reporta únicamente la latencia de inferencia de Tier 2 como resultado cuantitativo válido.
- **Tier 2 — resolución reducida.** La resolución de entrada se redujo de 320×320 (planeada) a 192×192 por límite de SRAM, lo que reduce la comparabilidad directa con Tier 3 (640×640) y la simulación.
- **Tier 1 — mAP no obtenido.** La biblioteca ArduTFLite usada en el firmware no exponía el tensor de salida INT8 cuantizado, lo que impidió calcular mAP tanto en la sesión de medición de latencia como en el barrido automatizado posterior sobre COCO128 (ver §4.2). No se pudo obtener una única sesión de datos que combine mAP y latencia para Tier 1.

- **Dataset reducido:** COCO128 (128 imágenes) tiene mayor varianza de mAP que COCO val (5000 imágenes).
- **Modelo de iluminación simplificado:** la corrección gamma no captura ruido de sensor, *lens flare* ni iluminación no uniforme.
- **CoreML bloqueado:** incompatibilidad de `coremltools` 9.0 con Python 3.12 impidió evaluar el Neural Engine de Apple.

5.9. Implicancias Prácticas

Los resultados tienen cuatro implicancias concretas para el despliegue de detección de objetos en edge:

1. **El preprocesamiento es crítico.** Dado que el punto de quiebre ocurre en $\gamma \approx 0,37$, una etapa de normalización adaptativa (CLAHE) podría extender la zona viable sin cambio de hardware.
2. **Un NPU dedicado es necesario pero no suficiente.** Reduce la brecha de latencia en casi dos órdenes de magnitud respecto a un MCU sin acelerador, pero no basta por sí solo para un modelo de 80 clases a la resolución evaluada.
3. **La latencia es independiente del gamma,** confirmado ahora en hardware real (Tier 3): la decisión de tier puede tomarse en gran medida con la restricción de mAP, simplificando el análisis de viabilidad operacional.
4. **El espacio de clases es una palanca de optimización subutilizada.** Si la aplicación final no requiere las 80 clases de COCO, reducir el espacio de salida es una vía plausible para que Tier 2 cruce el umbral de 50 ms sin cambiar de hardware.

6. Conclusiones

Este trabajo evaluó el impacto de la variación de iluminación, modelada mediante corrección gamma, sobre YOLOv8n en simulación y en los tres niveles de hardware de borde definidos originalmente, completando la evaluación de Tier 2 y Tier 3 que quedaba pendiente en el avance previo.

Hallazgos confirmados:

1. **H1 confirmada,** tanto en simulación como en hardware real (Tier 3). La degradación es asimétrica: $-21,9\%$ de mAP en sub-exposición ($\gamma=0,3$) vs $-3,8\%$ en sobre-exposición ($\gamma=1,8$). El punto de quiebre de viabilidad ocurre en $\gamma \approx 0,37$.
2. **80 % de clases colapsan.** 57 de 71 clases caen bajo el umbral de mAP en $\gamma=0,3$; las más afectadas son objetos pequeños con textura fina.
3. **El confidence score cae antes que el mAP.** El número de detecciones con $\text{conf} \geq 0,25$ cae $27,3\%$,

revelando que la degradación comienza en el clasificador.

4. **H2 parcialmente confirmada, con evidencia cuantitativa completa.** Ningún tier embebido cumple ambos umbrales de viabilidad, pero el dispositivo con NPU dedicado (Tier 2) reduce el exceso de latencia a $1,72\times$ el umbral, frente a $154\times$ (Tier 1) y $20,7\times$ (Tier 3) en dispositivos sin aceleración dedicada. La aceleración NPU es necesaria pero no suficiente para este modelo y dataset.
5. **Los resultados negativos son informativos.** La ausencia de un tier plenamente viable no invalida el estudio: cuantifica con precisión la distancia a la viabilidad de cada clase de hardware, información directamente accionable para decisiones de despliegue.

7. Trabajo Futuro

- Resolver el control de exposición manual en el WE2 (Tier 2) para obtener mediciones de mAP comparables con los demás tiers, cerrando el único punto de datos incompleto de este estudio.
- Evaluar Tier 2 con un subconjunto reducido de clases (p. ej., solo las relevantes a una aplicación de vigilancia o conteo) para verificar si la latencia cae bajo el umbral de 50 ms, siguiendo la hipótesis planteada en §5.6.
- Evaluar CLAHE como mitigación de preprocesamiento sin cambio de hardware, dado el punto de quiebre identificado en $\gamma \approx 0,37$.
- Extensión a otros modelos ligeros (EfficientDet-Lite, YOLOv5n) para comparación arquitectónica.
- Validación con iluminación física real (Fase B), en vez de corrección gamma sintética.

Agradecimientos

Al profesor Nicolás Torres por la guía académica y retroalimentación durante el desarrollo de este trabajo.

Referencias

- [1] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, 2019.
- [2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.
- [3] G. Jocher, A. Chaurasia, and J. Qiu, "YOLO by Ultralytics," <https://github.com/ultralytics/ultralytics>, 2023.
- [4] C. Banbury, V. J. Reddi, P. Torelli, J. Holleman, N. Jeffries, C. Kiraly, P. Montino, D. Kanter, S. Ahmed, D. Pau *et al.*, "MLPerf tiny benchmark," in *Proceedings of the Neural Information Processing Systems (NeurIPS) Track on Datasets and Benchmarks*, 2021, arXiv:2106.07597.
- [5] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.
- [6] A. Fanariotis, T. Orphanoudakis, K. Kotrotsios, V. Fotopoulos, G. Keramidas, and P. Karkazis, "Power efficient machine learning models deployment on edge IoT devices," *Sensors*, vol. 23, no. 3, p. 1595, 2023.
- [7] A. Fanariotis, T. Orphanoudakis, and V. Fotopoulos, "Evaluating the energy efficiency of NPU-accelerated machine learning inference on embedded microcontrollers," arXiv:2509.17533, 2025.
- [8] A. Dey, S. Srivastava, G. Singh, and R. G. Pettit, "Real-time performance benchmarking of TinyML models in embedded systems (PICO: Performance of inference, CPU, and operations)," arXiv:2509.04721, 2025.
- [9] D. Hendrycks and T. Dietterich, "Benchmarking neural network robustness to common corruptions and perturbations," in *International Conference on Learning Representations (ICLR)*, 2019.
- [10] C. Li, C. Guo, L. Han, J. Jiang, M.-M. Cheng, J. Gu, and C. C. Loy, "Low-light image and video enhancement using deep learning: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 12, pp. 9396–9416, 2021.
- [11] X. Xu, R. Wang, C.-W. Fu, and J. Jia, "SNR-aware low-light image enhancement," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 17 714–17 724.
- [12] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *European Conference on Computer Vision (ECCV)*, 2014, pp. 740–755.

Apéndices

A. Figuras Suplementarias

Las siguientes figuras complementan el análisis principal y se incluyen por completitud; no son centrales para la argumentación de H1/H2 pero respaldan el análisis explicativo detallado.

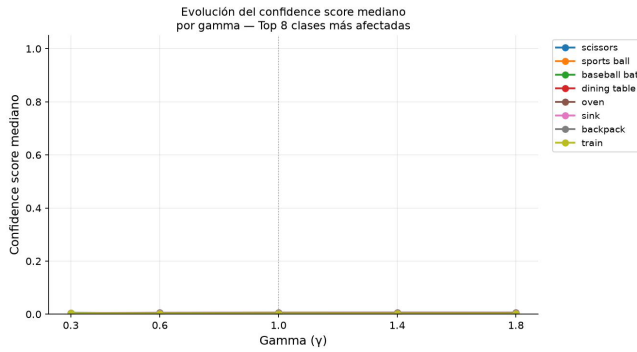


Figura 14: Evolución del confidence score mediano por gamma para las 8 clases más afectadas.

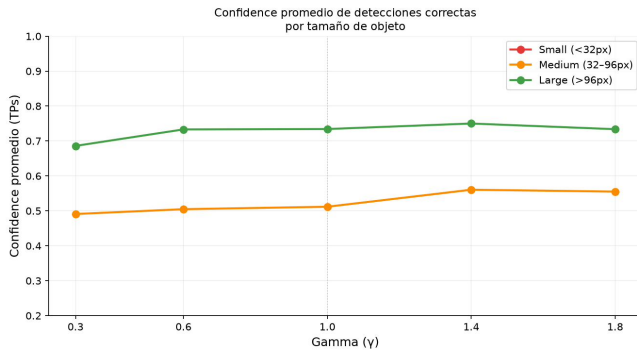


Figura 15: Confidence promedio de detecciones correctas (TPs) por tamaño de objeto y gamma.

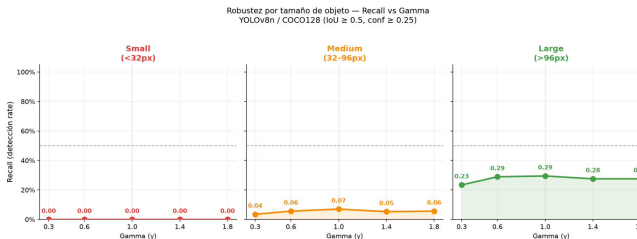


Figura 16: Robustez (recall) por tamaño de objeto y gamma, paneles separados por categoría de tamaño.

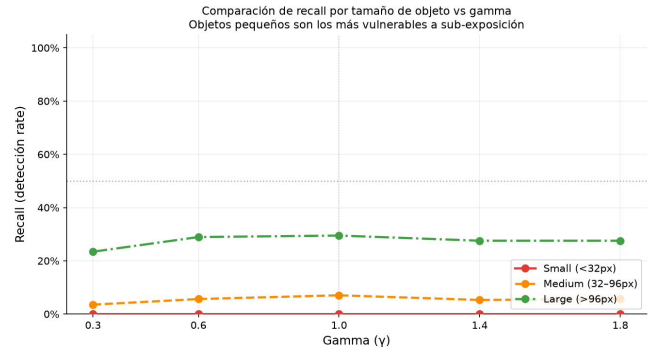


Figura 17: Comparación superpuesta de recall por tamaño de objeto vs gamma.

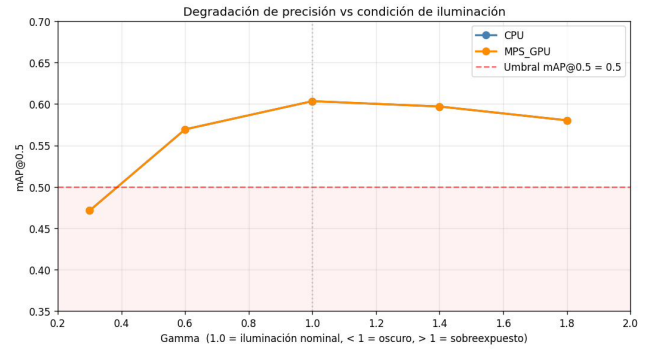


Figura 18: Degradación de precisión (mAP@0.5) vs condición de iluminación, modos CPU y MPS GPU superpuestos.

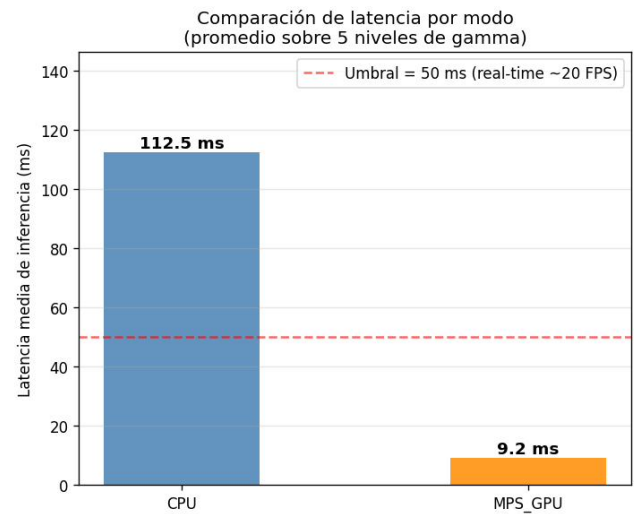


Figura 19: Comparación de latencia media de inferencia por modo (CPU vs MPS GPU), promediada sobre los 5 niveles de gamma.

B. Datos y Código

El código fuente, los notebooks de análisis y los datos crudos (`metrics.csv`, `tier1_*.json`, `tier2_latency.csv`, `tier2_raw_log.txt`, `tier3_results.json`, `confidence_scores_raw.csv`, `per_class_map_by_gamma.csv`, `size_fine_grid.csv`, `map_fine_grid.csv`) están disponibles en el repositorio del proyecto. Scripts relevantes: `export_tier2_model.py` (exportación del modelo a TFLite INT8 para el NPU Ethos-U55) y `tier2_latency_capture.py` (captura de latencia vía I2C/serial sobre el firmware WiseEye2).

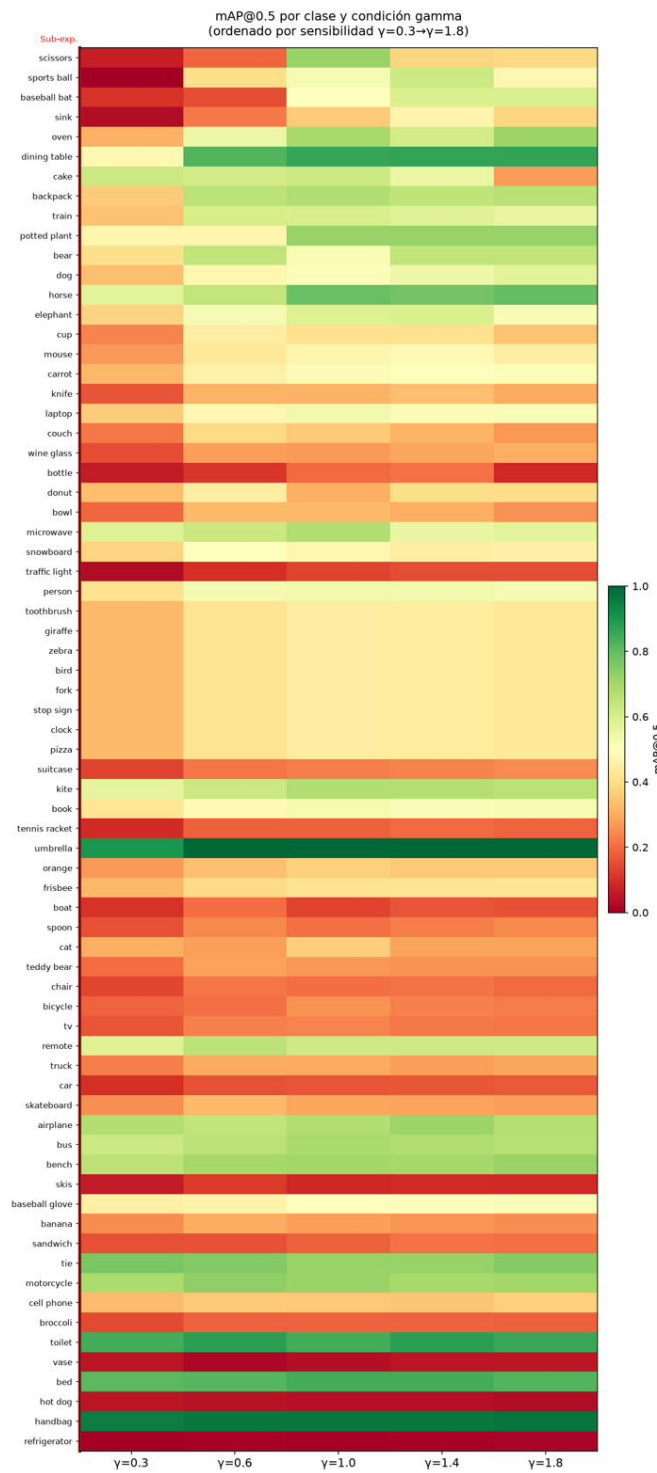


Figura 7: Heatmap de mAP@0.5 para las 71 clases de COCO128, ordenadas por sensibilidad a gamma. La columna $\gamma=0,3$ concentra la mayor densidad de celdas rojas, confirmando que la sub-exposición es el factor de mayor impacto.

Fallos de detección por clase — $\gamma=1.0$ vs $\gamma=0.3$
Clases más afectadas por sub-exposición

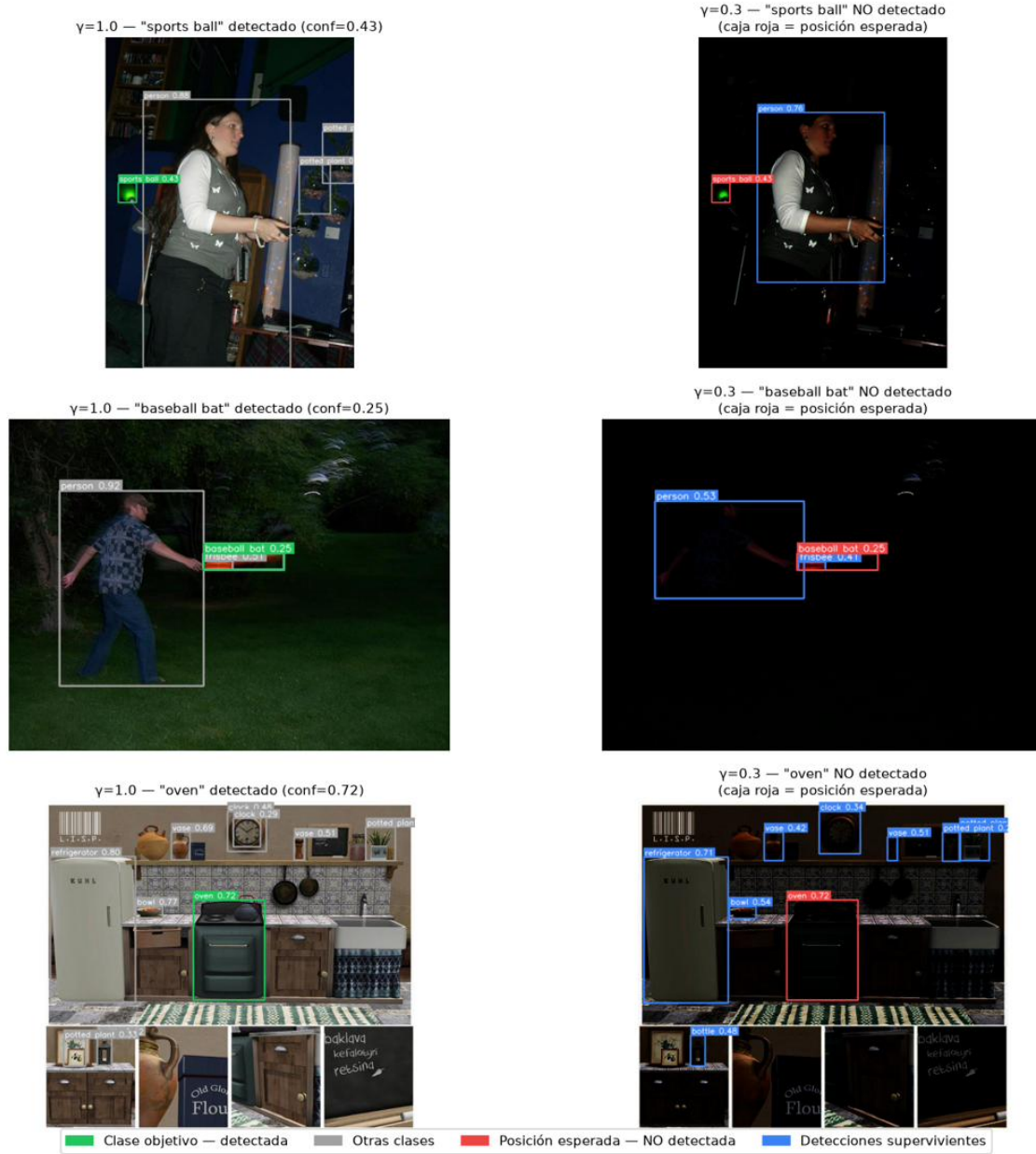


Figura 10: Fallos de detección por clase en $\gamma=0.3$. Verde: detectado en $\gamma=1.0$. Rojo: posición esperada no detectada en $\gamma=0.3$. Azul: detecciones supervivientes.

Visualización de detecciones fallidas — YOLOv8n bajo sub-exposición ($\gamma=0.3$)
Imágenes seleccionadas por mayor caída en número de detecciones

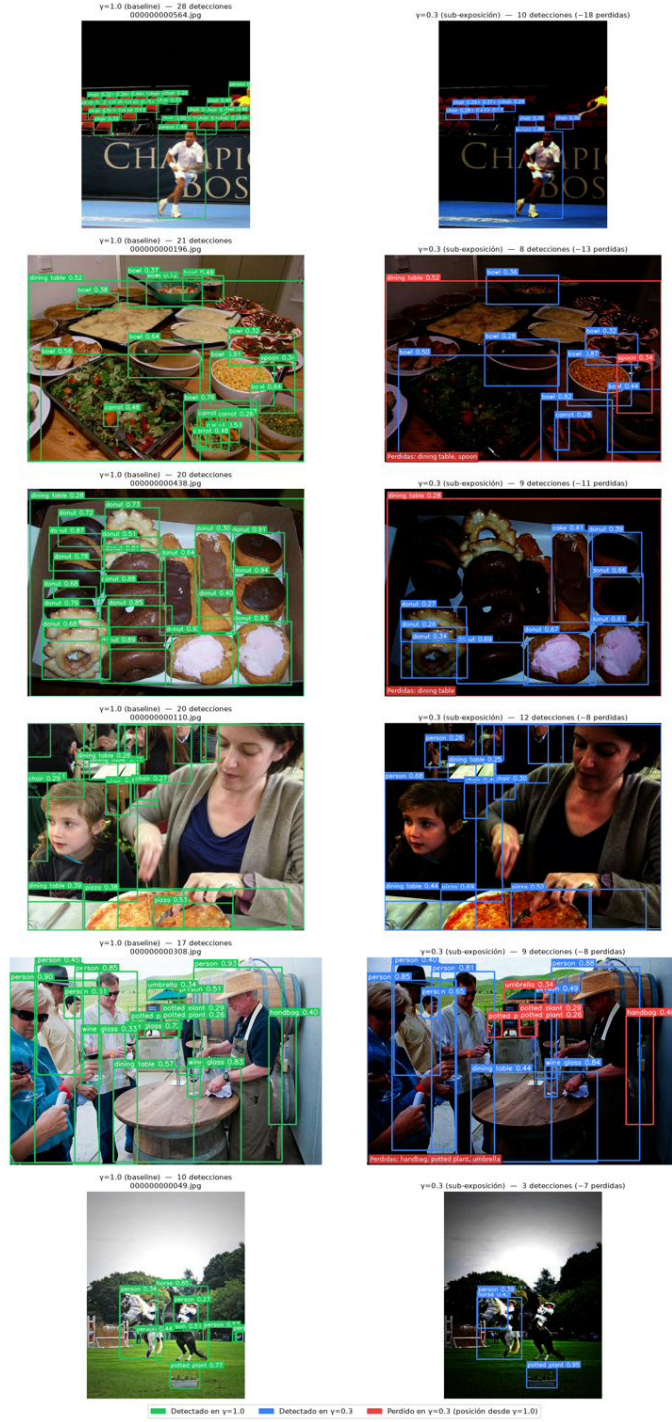


Figura 11: Comparación de detecciones en las 6 imágenes con mayor caída. Columna izquierda: $\gamma=1,0$ (baseline, cajas verdes). Columna derecha: $\gamma=0,3$ (cajas azules = supervivientes, cajas rojas = objetos perdidos).